

Relatorio Tecnico - Go-Back-N via UDP

Relatório Técnico — Implementação do Protocolo Go-Back-N via UDP em Java

Disciplina: Redes de Computadores

Autores: Felipe Araújo Correia, Luís Filipi Rosa dos Santos

1. Introdução

Este relatório descreve a implementação do protocolo **Go-Back-N (GBN)** utilizando **sockets UDP** em Java, conforme especificado no trabalho final da disciplina de Redes de Computadores. O UDP não oferece confiabilidade, ordenação nem controle de perdas, de modo que toda a lógica de transferência confiável foi construída na camada de aplicação, seguindo as máquinas de estado finitas apresentadas em Kurose & Ross.

O sistema é composto por dois módulos independentes: o **Emissor**, responsável por fragmentar o arquivo e gerenciar a janela deslizante, e o **Receptor**, que aceita apenas pacotes em ordem e simula perdas de acordo com uma probabilidade configurada.

2. Objetivos

- Implementar o protocolo Go-Back-N fiel às FSMs do livro-texto.
 - Garantir a transferência correta de arquivos binários sobre UDP.
 - Simular perdas de pacotes de forma controlada.
 - Exibir estatísticas detalhadas no emissor e no receptor.
 - Verificar a integridade do arquivo recebido por meio de hash MD5.
-

3. Arquitetura da Solução

3.1 Módulos

Módulo	Arquivo principal	Responsabilidade
Emissor	<code>Emissor.java</code> , <code>GBNSender.java</code> , <code>AckReceiver.java</code>	Envia pacotes, gerencia janela, trata timeouts e retransmissões
Receptor	<code>Receptor.java</code> , <code>GBNReceiver.java</code>	Recebe pacotes, descarta fora de ordem, simula perdas, salva arquivo
Protocolo	<code>Packet.java</code> , <code>PacketType.java</code> , <code>ProtocolConfig.java</code>	Formato dos datagramas e constantes
Utilitários	<code>FileUtils.java</code> , <code>HashUtils.java</code> , <code>ArgumentParser.java</code>	Leitura/escrita de arquivos, MD5 e parsing de argumentos

3.2 Formato do Datagrama

Cada datagrama possui um cabeçalho de 11 bytes seguido do payload:

Campo	Tamanho	Descrição
Tipo	1 byte	0=DATA, 1=ACK, 2=HANDSHAKE, 3=FIN
Número de sequência	4 bytes	int — identificador do pacote
Número de ACK	4 bytes	int — confirmação cumulativa
Tamanho dos dados	2 bytes	short — bytes válidos no payload
Dados	até 1024 bytes	payload do arquivo

3.3 Máquinas de Estado Finitas

Emissor: a implementação utiliza duas threads concorrentes: a thread principal envia novos pacotes enquanto houver espaço na janela N, e uma thread `AckReceiver` processa os ACKs recebidos continuamente. Um temporizador dedicado (`ScheduledExecutorService`) é iniciado para o pacote mais antigo não confirmado (`base`) e é reiniciado toda vez que um ACK cumulativo avança a base.

Receptor: o receptor mantém um único estado com `expectedseqnum`. Ao receber um pacote DATA: - se `seq == expectedseqnum`, o pacote é aceito (sujeito à simulação de perda) e um ACK é enviado; - caso contrário, o pacote é descartado e o último ACK válido é reenviado.

4. Implementação

4.1 Emissor (`GBNSender.java`)

As principais variáveis do protocolo são:

- **base**: número de sequência do pacote mais antigo não confirmado.
- **nextSeqNum**: próximo número de sequência a ser usado.
- **windowSize (N)**: tamanho da janela de transmissão.

Thread principal:

```
enquanto base < totalPacotes:
    enviar novos pacotes enquanto nextSeqNum < base + N
    dormir 10 ms
```

Thread AckReceiver:

```
enquanto em execução:
    receber datagrama
    se for ACK e ackNumber >= base:
        base = ackNumber + 1
        reiniciar ou cancelar o temporizador
```

Temporizador:

```
se timeout ocorrer:
    retransmitir todos os pacotes de base até nextSeqNum - 1
    reiniciar o temporizador
```

O emissor finaliza o envio com um pacote FIN e aguarda um ACK do FIN, retransmitindo-o até cinco vezes se necessário.

4.2 Receptor (GBNReceiver.java)

O receptor aguarda um pacote HANDSHAKE contendo probabilidade de perda, caminho do arquivo de destino, tamanho do arquivo e hash MD5 original. A partir daí, entra no loop principal de recepção.

A simulação de perda atua **somente sobre pacotes recebidos em ordem**, conforme exigido no trabalho. A geração de números aleatórios utiliza `Random.nextDouble() < lossProbability` para decidir o descarte silencioso.

4.3 Estatísticas

Emissor: - Pacotes enviados (incluindo retransmissões) - Retransmissões - ACKs recebidos - Bytes enviados - Tempo total e throughput estimado

Receptor: - Pacotes DATA recebidos - Pacotes em ordem recebidos - Perdas artificiais - Pacotes fora de ordem descartados - Taxa de perda efetiva (perdas / pacotes em ordem)

A taxa de perda efetiva é calculada apenas sobre os pacotes recebidos em ordem, para que, pelo princípio da Lei dos Grandes Números, tenda à probabilidade configurada.

5. Testes Realizados

5.1 Ambiente de Teste

- **Sistema operacional:** Linux
- **Linguagem:** Java (JDK padrão)
- **Execução:** localhost (127.0.0.1)
- **Arquivo de teste:** arquivo_1mb.bin (1.124.800 bytes, 1,1 MB)
- **Tamanho do payload:** 1024 bytes
- **Total de pacotes de dados:** 1100
- **Timeout:** 500 ms

5.2 Casos de Teste

Foram executados os seguintes cenários:

1. Transferência sem perdas (**prob_perda** = 0.0), janela N=8.
2. Transferência com 10% de perda simulada (**prob_perda** = 0.10), janela N=8.
3. Comparação de desempenho para N = 4, 8 e 16, com 10% de perda.
4. Transferência utilizando porta customizada (5001).
5. Transferência do arquivo de exemplo **arquivo_grande.txt**.

5.3 Resultados

Cenário 1 — 0% de perda, N = 8

Métrica	Emissor	Receptor
Pacotes enviados	1100	1100
Retransmissões	0	—
ACKs recebidos	1100	—
Perdas artificiais	—	0
Fora de ordem	—	0
Taxa de perda	—	0,00%
Tempo total	2394 ms	—
Throughput	470.509,61 bytes/s	—
Integridade	OK (MD5 igual)	OK

Cenário 2 — 10% de perda, N = 8

Métrica	Emissor	Receptor
Pacotes enviados	~2020	~2020
Retransmissões	~920	—
ACKs recebidos	1100	—
Perdas artificiais	—	~115

Métrica	Emissor	Receptor
Fora de ordem	—	~805
Taxa de perda	—	~10,45%
Tempo total	~59.564 ms	—
Throughput	~34.727 bytes/s	—
Integridade	OK (MD5 igual)	OK

Cenário 3 — Comparação por tamanho da janela (10% de perda)

Janela N	Tempo (ms)	Throughput (bytes/s)	Retransmissões	Perdas artificiais	Taxa efetiva
4	65.237	25.051,80	496	124	11,27%
8	59.564	34.727,02	920	115	10,45%
16	61.166	49.554,33	1860	119	10,82%

Observação: o throughput mostrado no emissor considera todos os bytes transmitidos, incluindo retransmissões. A taxa de transferência útil (arquivo / tempo) é similar entre N=8 e N=16, mas o número de bytes retransmitidos cresce com a janela.

Cenário 4 — Porta customizada (5001)

Métrica	Valor
Comando receptor	<code>java -cp bin Receptor 5001</code>
Comando emissor	<code>java -cp bin Emissor ... 0.10 5001</code>
Integridade	OK (MD5 igual)
Taxa de perda efetiva	11,36%

Cenário 5 — Arquivo de texto (arquivo_grande.txt)

Métrica	Emissor	Receptor
Pacotes enviados	600	600
Retransmissões	309	—
ACKs recebidos	291	—
Perdas artificiais	—	40
Taxa de perda	—	13,75%
Integridade	OK (MD5 igual)	OK

6. Análise dos Resultados

6.1 Correção do Protocolo

Em todos os cenários o arquivo recebido teve o mesmo hash MD5 do arquivo original, o que demonstra que o Go-Back-N está recuperando corretamente as perdas simuladas. O receptor descarta pacotes fora de ordem e reenvia ACKs cumulativos, enquanto o emissor retransmite toda a janela a partir de **base** no timeout.

6.2 Impacto da Janela N

Os resultados mostram que:

- **N=4** apresentou o menor throughput e o maior tempo de conclusão. A janela pequena limita o pipeline, fazendo com que o emissor fique ocioso à espera de ACKs.
- **N=8** equilibrou tempo de conclusão e número de retransmissões, sendo uma escolha razoável para 10% de perda.
- **N=16** aumentou o throughput bruto (bytes enviados / tempo), mas também aumentou significativamente as retransmissões. Isso ocorre porque, quando um pacote se perde, toda a janela é retransmitida; janelas maiores geram mais retransmissões desnecessárias.

6.3 Taxa de Perda Efetiva

A taxa de perda efetiva ficou próxima dos 10% configurados nos cenários com maior número de pacotes (arquivo de 1,1 MB). No arquivo menor (**arquivo_grande.txt**), a variância foi maior (13,75%), o que é esperado devido ao menor tamanho da amostra. Isso confirma o comportamento predito pela Lei dos Grandes Números.

6.4 Temporizador e Concorrência

A mudança para uma arquitetura com duas threads e um **ScheduledExecutorService** permitiu que o emissor processasse ACKs enquanto continuava a enviar novos pacotes. Isso eliminou o comportamento anterior de “envia janela, espera ACK”, aproximando a implementação da FSM do GBN descrita no livro.

7. Conclusão

As dificuldades encontradas incluíram a coordenação entre as threads do emissor e o gerenciamento do temporizador, além de garantir que o pacote FIN fosse confiavelmente reconhecido. A solução adotada foi a separação do envio e da recepção de ACKs em threads distintas, sincronizadas por meio de **synchronized**, e a adição de um handshake de FIN com retransmissão.

Como melhoria futura, pode-se investigar a adoção de um buffer circular de tamanho N (conforme sugerido no trabalho) para reduzir o consumo de memória em arquivos muito grandes, e a implementação do controle de estouro do espaço de números de sequência para transferências acima de 2 GB.

Referências

KUROSE, James F.; ROSS, Keith W. *Redes de Computadores: Uma Abordagem Top-Down*. 8. ed. São Paulo: Pearson, 2021. Capítulo 3, Seções 3.4 e 3.4.3.